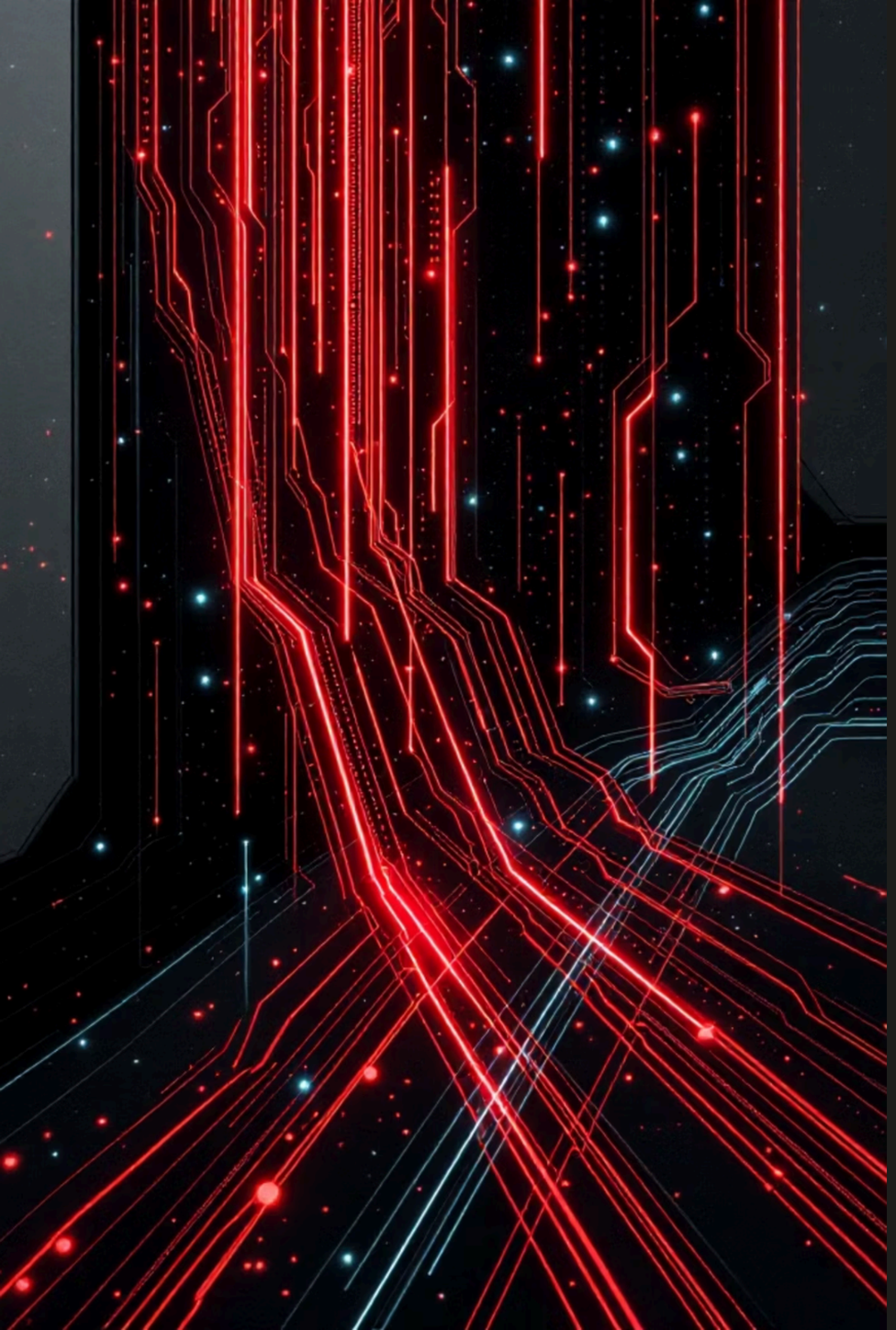


Rust: Segurança na Programação

Modelo de Ownership: Referências e Borrowing

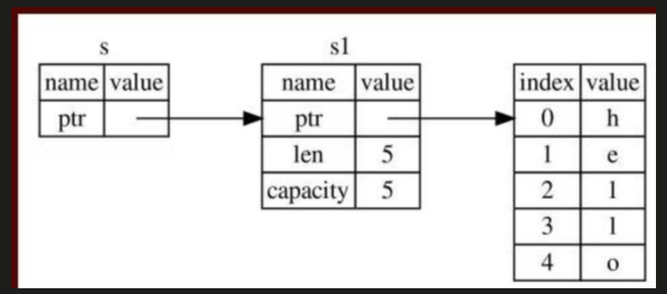
NUNO NETO · 2026

CONNECT5
EDIH European Digital Innovation Hubs Network
U.PORTO FACULDADE DE CIÊNCIAS UNIVERSIDADE DO PORTO
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
PRR Plano de Recuperação e Resiliência

Referências

Para resolver os problemas do modelo de ownership, Rust introduz as **referências** — endereços de memória que permitem aceder a dados sem tomar a sua posse.

> Quando uma referência é destruída, o objeto referido **não é destruído**.



Em Rust, as referências apontam sempre para um valor **válido**, do tipo do dono e dentro da *lifetime* do dono — eliminando os riscos clássicos dos pointers.

As referências são tipos primitivos que implementam **Copy**, comportando-se como os tipos escalares primitivos.

Sintaxe de Referências

1

Aceitar como argumento

```
fn tamanho_str(s: &String)
-> usize {
    s.len()
}
```

O **&** antes do tipo indica que recebemos uma referência.

2

Criar uma referência

```
let objeto: String =
String::new();
let referencia: &String =
&objeto;
```

referencia aponta para *objeto* mas **não é dona** do objeto.

3

Inserção automática

O Rust insere automaticamente a referência em muitas situações, como na chamada a string.len(), onde &self é passado implicitamente.

Desreferenciar Objetos

Com o operador * obtemos o objeto referenciado diretamente — **dereferencing**.

Tipo Copy (i32)

```
let test_: i32 = 10;
let test_ref: &i32 = &test_;
let test_2: i32 = *test_ref;
```

Copia o valor — funciona sem restrições.

Tipo não-Copy (String)

```
let str: String = String::new();
let str_ref: &String = &str;
let str_2: String = *str_ref;
```

O move apenas tem sucesso em casos específicos, como mover para fora de um *Box*.

Borrowing: Implicações

O ato de criar uma referência chama-se **empréstimo** (*borrowing*). Enquanto existe um empréstimo ativo, a variável original fica restrita.

Código que falha

```
let mut var: String = String::new();
let var_ref: &String = &var;
var = String::from("Ola"); //
```

Referências Mutáveis

As referências são **imutáveis por defeito**. Para alterar valores sem transferir ownership, usam-se **referências mutáveis**.

Criar referência mutável

```
let mut obj: String =
String::new();
let mut_ref: &mut String = &mut
obj;
*mut_ref += " ";
```

Apenas objetos **mutáveis** podem ter referências mutáveis.

Alterar conteúdo

```
let mut array: [i32;5] =
[1,2,3,4,5];
let array_ref: &mut [i32;5] =
&mut array;
array_ref[0] = 10;
```

Substituir valor total

```
let mut array: [i32;5] =
[1,2,3,4,5];
let array_ref: &mut [i32;5] =
&mut array;
*array_ref = [6, 7, 8, 9, 10];
```

Apenas pode existir **uma** referência mutável — ou qualquer número de imutáveis — para um objeto em simultâneo. Isto elimina *data races* estaticamente.

Lifetimes de Referências

Uma referência tem **sempre** que viver **menos tempo** que o objeto que a criou. Caso contrário, obteríamos uma referência inválida o que levaria a comportamento indefinido.

Inválido ❌

```
let str_ref: &String = {
    let string = String::from("Olá!");
    &string // string é destruída aqui
};
```

```
error[E0597]: `string` does not
live long enough
```

Válido ✅

```
// CORRETO: referência passada
// como argumento tem lifetime
// maior que a função
fn test_func2(str: &String)
    -> &String { str }
```

A referência é *owned* pelo *caller*, logo a sua *lifetime* é automaticamente maior que a da função.

Slices

Slices referenciam uma sequência contígua de elementos de uma coleção **sem tomar a sua ownership**.

Sintaxe

```
let array: [i32;7] =
[1,2,3,4,5,6,7];
let slice: &[i32] =
&array[1..3];
let slice_2: &[i32] =
&array[3..6];
```

Tipo e Comportamento

Tipo: &[<tipo>] (ex: &[i32]). Funcionam como referências — iteráveis com **for**, acessíveis por índice com **[]**. Não é necessário conhecer o tamanho estaticamente.

Eficiência

Apontam diretamente para o objeto na heap. Sem capacidade armazenada, pois a slice **não permite alterar a forma do objeto original**.

Composição e Utilidade das Slices

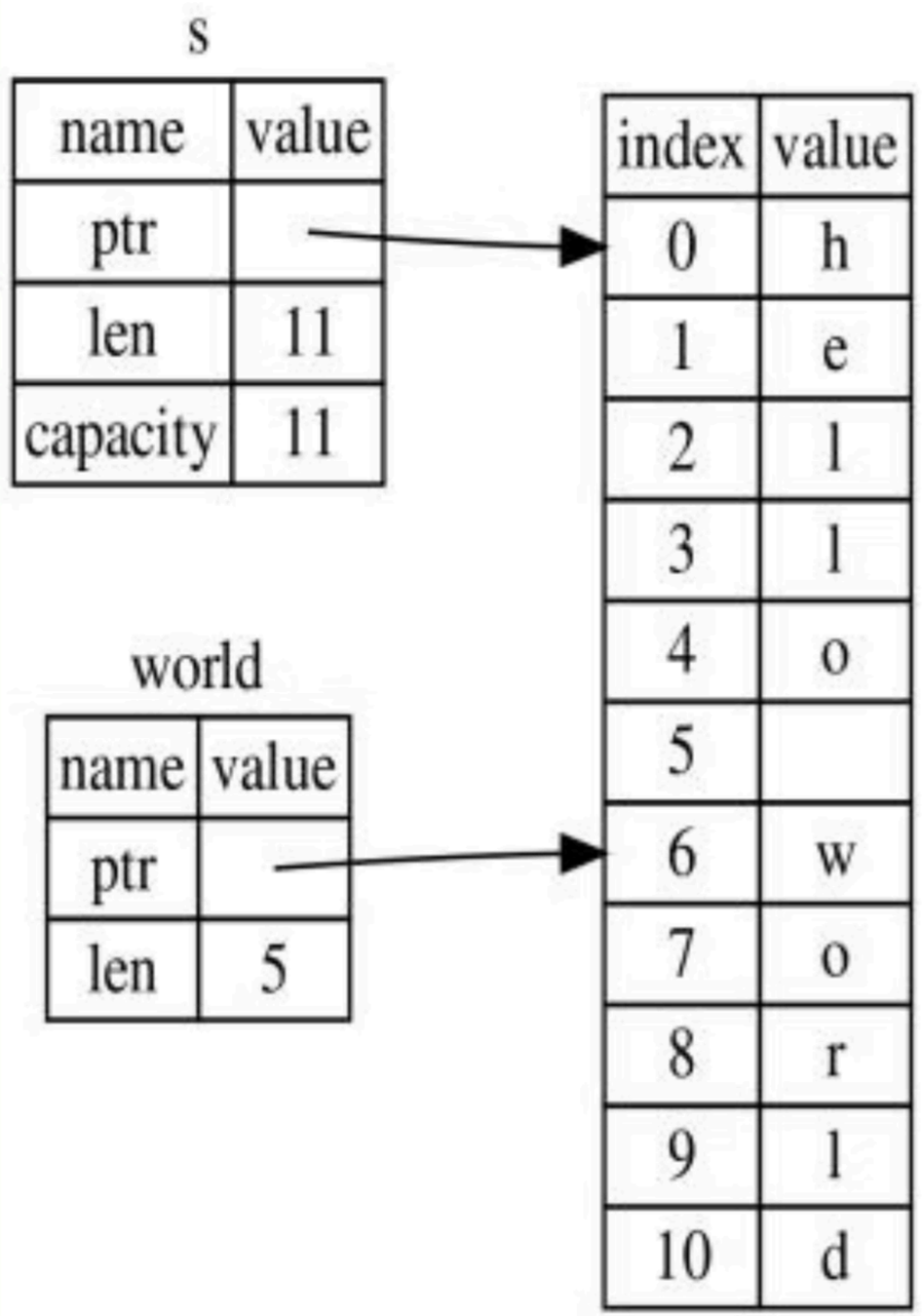
Duas slices podem apontar para a mesma coleção subjacente em posições diferentes — sem copiar dados.

Função com array fixo

```
fn aceita_colecao(array: [i32; 5]) {
    for i in array { println!("{}", i); }
}
```

Limitado a arrays de tamanho 5.

Função com slice

```
fn aceita_colecao_slice(array: &[i32]) {
    for i in array { println!("{}", i); }
}
```

Aceita **qualquer** coleção — mais **flexível** e **eficiente**.